

CS Lab — End-sem Practice

OOPS Lab (probably harder)

Time Limit: **2 seconds per problem** Language: **C++17** Compiler flag: **-std=c++17**

Instructions: This lab contains two independent problems. Each problem must be implemented as a C++ class with the interface specified.

All problems are handled in a **single program**. The first line of input is a **problem indicator** (1 or 2) that determines which problem to execute.

After reading the problem-specific input, the program may process additional commands (if applicable) until the keyword **end** is encountered.

Solutions that exceed the time limit will receive zero marks, regardless of correctness.

Input Routing:

```
1  int main() {
2      int problem;
3      cin >> problem;
4
5      if (problem == 1) {
6          // CSV Analyzer
7          // read input, construct objects, process queries
8          return 0;
9      }
10
11     if (problem == 2) {
12         // Shortest Path (Skip List + Dijkstra)
13         // read graph input, compute results, print output
14         return 0;
15     }
16 }
```

Note:

- Each problem has its own input format and output requirements.
- Follow the exact output format specified in each section.
- Do not print any extra text or prompts.

Test Case Distribution

The evaluation consists of a total of **50 test cases**:

- **30 test cases** for Problem 1 (CSV Analyzer)
- **20 test cases** for Problem 2 (Graph / Dijkstra)

This corresponds to a **60% – 40% distribution** across the two problems.

Type-Aware CSV Analyser

Problem Statement

You are given a CSV file where the first row specifies the type of each column. Each column can be of type `int`, `double`, or `string`.

You must design a system using templates and object-oriented programming to:

- Represent each column using a templated class `CSVColumn<T>`
- Store all columns inside a container class `CSVData`
- Support queries of the form:

`query_stats <column_index>`

Query behaviour:

- If the column is numeric $\rightarrow$ print min, max, mean, variance
- If the column is `string` $\rightarrow$ throw an exception

Example Class Interface

You are free to add any extra member variables or functions as you desire.

```
1  // Base class
2  class BaseColumn {
3  public:
4      virtual void printStats() const = 0;
5      virtual ~BaseColumn() = default;
6  };
7
8  // Templated column
9  template <typename T>
10 class CSVColumn : public BaseColumn {
11 private:
12     std::vector<T> data;
13
14 public:
15     void addValue(const T& val);
16
17     T minVal() const;
18     T maxVal() const;
19     double mean() const;
20     double variance() const;
21
22     void printStats() const override;
23 };
24
25 // Container
26 class CSVData {
27 private:
28     std::vector<BaseColumn*> columns;
29
30 public:
31     void addColumn(BaseColumn* col);
32     void queryStats(int columnIndex) const;
33 };
```

Class Requirements

- `CSVColumn<T>` must store data using `vector<T>`
- Statistical functions are valid only for numeric types
- Calling stats on a string column must throw an exception
- Output values must be printed with **2 decimal precision**

Variance is defined as:

$$\text{Var}(x) = \frac{\sum x^2}{n} - \left(\frac{\sum x}{n} \right)^2$$

Implementation Hint (Optional)

When implementing `printStats`, note that numeric and string columns require different behavior, specifically templating will fail when we try to cast string to double to solve for the stats.

One approach is to use **function overloading** on a helper function: define a general version for numeric types and a separate overload for `string` that throws an exception.

```

1  // generic version (for numeric types)
2  template<typename T>
3  void printStatsImpl(int idx, const vector<T>& data) {
4      // compute min, max, mean, variance
5  }
6
7  // overload for string
8  void printStatsImpl(int idx, const vector<string>&) {
9      throw runtime_error("ERR_TYPE");
10 }
```

Alternative: You may also use compile-time type checks such as `if constexpr` together with `std::is_same<T, string>`.

- `std::is_same<T, U>` is a type trait that evaluates to true if the two types are identical.
- `if constexpr` is a compile-time conditional: only the selected branch is compiled.

This allows you to write different logic for string and numeric types while avoiding invalid operations (e.g., converting a string to a number).

```

1  if constexpr (std::is_same<T, string>::value) {
2      throw runtime_error("ERR_TYPE");
3  } else {
4      // numeric computation
5  }
```

Exception Specifications

Define a custom exception hierarchy. Each exception must implement a `what()` method returning the exact string below.

Exception Class	Triggering Condition	Output
FileNotFoundException	CSV file cannot be opened	ERR_FILE
MalformedRowException	Row has incorrect number of columns	ERR_ROW
MalformedCellException	Cell cannot be parsed to declared type	ERR_CELL
InvalidColumnIndex	Query index out of bounds	ERR_INDEX
InvalidColumnType	Stats requested on string column	ERR_TYPE

Notes:

- All exceptions must inherit from `std::exception`
- Output must match exactly (no extra text)
- If an error occurs during parsing:
 - Print the corresponding `ERR_*` code to **stdout**
 - Skip the faulty row, you may need to **remove previously added columns data**.
 - Continue processing remaining rows
- If an error occurs during a query:
 - Print the `ERR_*` code
 - Continue to next query

Input Format

- First line: CSV filename/path
- Remaining lines:
 - `query_stats <N>`
 - `end`

Output Format

- Numeric column:

```
Col <i>: min=<v> max=<v> mean=<v> var=<v>
```

- Error:

```
ERR_*
```

Sample Test Cases**Test 1 — Normal case**

csv/data1.csv

```
int,string,double
1,Alice,3.85
2,Bob,3.60
3,Carol,3.95
```

Input

```
1
csv/data1.csv
query_stats 0
query_stats 1
query_stats 2
end
```

Output

```
Col 0: min=1.00 max=3.00 mean=2.00 var=0.67
ERR_TYPE
Col 2: min=3.60 max=3.95 mean=3.80 var=0.02
```

Test 2 — Multiple errors**csv/data2.csv**

```
int,double
1,2.0
2,abc
3
4,5.0
```

Input

```
1
csv/data2.csv
query_stats 1
end
```

Output

```
ERR_CELL
ERR_ROW
Col 1: min=2.00 max=5.00 mean=3.50 var=2.25
```

Test 3 — Invalid column index**Input**

```
1
csv/data1.csv
query_stats 5
end
```

Output

```
ERR_INDEX
```

Constraints

- $1 \leq \text{columns} \leq 20$
- $1 \leq \text{rows} \leq 10^5$
- Time limit: 2 seconds

Shortest Paths via a Skip-List Priority Queue

Problem Statement

Given a weighted directed graph with non-negative edge weights and a source vertex, compute the shortest path distance from the source to every vertex.

Vertices that are not reachable from the source should have distance `INF`.

You must implement Dijkstra's algorithm using a custom skip-list-based priority queue. Use of `std::priority_queue` or any other library priority queue is not allowed.

Required Class Interface

```

1  //          Skip List node

2  struct SLNode {
3      int key;
4      int vertex;
5      std::vector<SLNode*> forward;
6
7      SLNode(int key, int vertex, int height)
8          : key(key), vertex(vertex), forward(height, nullptr) {}
9  };
10
11 //          Skip List Priority Queue

12 class SkipListPQ {
13 private:
14     int maxLevel;
15     int curLevel;
16     SLNode* head;
17
18     std::mt19937 rng;
19
20     int randomLevel();
21
22 public:
23     explicit SkipListPQ(int n, int seed);
24     ~SkipListPQ();
25
26     void insert(int vertex, int dist);
27     std::pair<int, int> extractMin();
28     bool isEmpty() const;
29 };
30
31 //          Graph + Dijkstra

32 class ShortestPathGraph {
33 private:
34     int V;
35     std::unordered_map<int, std::vector<std::pair<int, int>>> adj;
36     std::unordered_map<int, int> dist;
37     std::unordered_map<int, int> pred;
38

```

```

39 public:
40     ShortestPathGraph(int V);
41
42     void addEdge(int u, int v, int w);
43     void computeShortestPaths(int source);
44
45     void printDistances() const;
46     void printPath(int source, int target) const;
47 };

```

Deterministic Behaviour Requirement

The skip list must be constructed deterministically.

- Use a fixed-seed random number generator
- Initialise using: `std::mt19937 rng(seed);`
- All level generation must use this generator

Using the Random Generator

Initialise the random number generator in the constructor using the provided seed:

```

1 SkipListPQ::SkipListPQ(int n, int seed)
2     : maxLevel(ceil(log2(n))), curLevel(0),
3       head(new SLNode(INT_MIN, -1, maxLevel)),
4       rng(seed) {}

```

Use this generator inside `randomLevel()`:

```

1 int SkipListPQ::randomLevel() {
2     int level = 0;
3     while ((rng() & 1) && level < maxLevel - 1)
4         level++;
5     return level;
6 }

```

Notes:

- Always use `rng()` for randomness
- Do not use `rand()`
- The same seed must produce identical skip list structure

Pseudocode

SkipListPQ::insert(vertex, dist)

```

if vertex already present with smaller or equal dist:
    return

find update[] array by traversing levels top-down

level = randomLevel()
create new node

for i from 0 to level:
    splice node into level i using update[i]

```

```
store/update best distance for vertex
```

SkipListPQ::extractMin()

```
node = head->forward[0]    // smallest element

for each level:
    if head->forward[level] == node:
        remove node from that level

delete node
return (dist, vertex)
```

Dijkstra's Algorithm

```
for each vertex v:
    dist[v] = INF

dist[source] = 0

pq = SkipListPQ(V, seed)
pq.insert(source, 0)

while pq is not empty:
    (d, u) = pq.extractMin()

    if d > dist[u]:
        continue

    for each (v, w) in adj[u]:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            pred[v] = u
            pq.insert(v, dist[v])
```

Design Notes

- The head node must have key INT_MIN
- The minimum element is always at level 0
- Graph edge weights are non-negative
- If two nodes have equal distance, prefer the smaller vertex ID

Input Format

- First line: two integers V and E
- Next E lines: three integers u v w
- Next line: source vertex
- Next line: target vertex

Output Format

- Print distances for all vertices in the format:

```
vertex : distance
```

- Print the shortest path from source to target:

```
u -> v -> ... -> t
```

- If unreachable, print:

```
No path
```

Sample Test Cases

Test 1

Input

```
2
5 7
0 1 10
0 2 3
1 3 2
2 1 4
2 3 8
2 4 2
3 4 5
0
4
```

Output

```
0 : 0
1 : 7
2 : 3
3 : 9
4 : 5

0 -> 2 -> 4
```

Test 2

Input

```
2
4 3
0 1 5
1 2 3
2 3 7
0
3
```

Output

```
0 : 0
1 : 5
2 : 8
3 : 15

0 -> 1 -> 2 -> 3
```

Test 3

Input

```
2
4 2
0 1 4
0 2 9
0
3
```

Output

```
0 : 0
1 : 4
2 : 9
3 : INF

No path
```

Test 4

Input

```
2
1 0
0
0
```

Output

```
0 : 0

0
```

Constraints

- $1 \leq V \leq 10^4$
- $0 \leq E \leq 5 \times 10^4$
- $0 \leq w \leq 10^6$
- Time limit: 2 seconds

End of Lab — Sorry and Good luck!